I use a mix of Claude, Codex, and Grok depending on the task, and I'll often have them check each other's work. The problem with juggling tools like that is each one starts every session with a blank memory. Something I teach one tool rarely carries over to another, and keeping them in sync across a week of work meant constantly re-explaining the same context.

Agent Hub is how I fixed that. It's a single git repository that acts as shared long-term memory across every agent I use and every device I use them from. Any agent that can read files can read it. Any agent that can run git can write to it. Because it's a normal GitHub repo, it's also there on my phone, not just my laptop.

What it actually is

It's not an app, and it's not a service. It's a folder of markdown files, structured like a wiki, pushed to GitHub. That's the whole mechanism. The intelligence is a small set of conventions every agent gets told to follow, written down once in a file at the root.

The structure

```
agent-hub/
├── AGENTS.md          # the contract: every agent reads this first
├── CLAUDE.md          # symlink to AGENTS.md (Claude Code's expected
filename)
├── HOME.md            # the index: start-here map of everything
├── projects/          # one page per ongoing initiative, current state
only
├── topics/            # cross-cutting concepts referenced by multiple
projects
├── docs/               # finished artifacts (memos, specs, decision
docs)
├── work/<project>/    # active working files, born here, committed as
they're made
│   └── private/       #   gitignored: sensitive stuff, local only,
never pushed
├── log/                # append-only journal, one dated file per
completed task
├── inbox/              # quick-capture drops (from mobile, usually),
waiting to be filed
├── .agents/skills/    # tool-neutral skills folder (Codex reads it);
```

```
the write-back skill lives here
└── .claude/ .codex/ .cursor/   # each tool's native config, pointing at shared conventions
```

Pages are wiki-style: short, and cross-linked with `[[double-bracket-links]]` (Obsidian's syntax, which is also how I browse the repo visually). A link resolves to a filename anywhere in the repo, so an agent can follow just the threads relevant to its task instead of reading everything.

The two protocols: this is the actual "how"

Reading, at the start of any task: 1. Read `HOME.md`. 2. Follow only the `[[links]]` relevant to the task at hand. 3. Don't bulk-read the whole repo. The link graph exists precisely so an agent loads cheap, targeted context instead of everything.

Writing, at the end of any meaningful task: 1. Log what happened to `log/YYYY-MM-DD-short-slug.md`: what was done, the key decisions and why, what was produced and where. 2. Update the relevant `projects/` page to reflect the new current state (the log holds history, the project page holds the present). 3. Link liberally. First mention of anything gets `[[linked]]`, even a page that doesn't exist yet (a "red link" just marks something worth writing later). 4. Update `HOME.md` if a new page got created. 5. Commit with a `log: <what happened>` message and push, after asking. *Decision (2026-08-30 → 2026-10-09, superseded): the original version of this step committed and pushed without asking. Several tools write to the one repo now, and the global pointer further down already said "ask first," so the contract caught up with the pointer.*

Every agent, whichever tool it is, gets told to do the same read and write dance. That's what makes the memory shared instead of per-tool.

Staying honest about what's still true

Markdown drifts out of date, so pages tag individual facts with freshness labels instead of trusting one "last updated" date for the whole page:

- `Current as of YYYY-MM-DD:` for an active fact or plan
- `Decision (YYYY-MM-DD):` for a settled choice, with the reason
- `Open question as of YYYY-MM-DD:` for something still unresolved, with an owner if I know one
- `Historical note (YYYY-MM-DD):` for context that shouldn't drive current work
- `Needs verification:` for something that may be stale; an agent checks before using it, and it's what the 30-day rule below turns a fact into

A label with only a start date tells you when something became true, not whether it still is. So a closed fact gets an end date and a pointer to what replaced it, instead of getting deleted or silently rewritten:

```
Decision (2026-06-01 → 2026-08-10, superseded by [[new-page]]):
```

A closed range means: don't act on this, follow the link for what's current now. An open range (start → present, or just a bare date) means: still active, but re-verify past 30 days. It's the same trick relational databases use for slowly changing history, a valid-from and a valid-to. It turns "is this still true?" from a judgment call into something an agent, or I, can resolve just by reading the label.

Keeping it tidy: the librarian

There's a small maintenance role, a "librarian," defined once and given to every tool (a Claude Code subagent, a Codex agent config, whatever the tool supports). On request ("maintain the hub"), it fixes broken links, files inbox items into the right pages, splits pages that have grown too long into a distilled current page plus a log entry, refreshes the `HOME.md` index, and flags stale projects. This is what keeps a wiki that several agents write into from turning into a junk drawer.

Keeping agents disciplined

"The agent will follow the rules" isn't a control, it's a hope. A written protocol only holds if something actually enforces it. Here's what does, in order of how much I trust it:

1. **There's no bulk-read affordance in the first place.** Reads go through individual file tools, one link at a time. There's no "dump the whole wiki" script or skill, and I won't build one later for convenience. That would undo the whole point. 2. **A stale or thin `HOME.md` is the real cause of over-reading**, not disobedience. An agent hedges and reads broadly when the index doesn't obviously answer "where do I look." Keeping `HOME.md` accurate is the load-bearing mechanism, not routine hygiene. 3. **The page-size cap is mechanical, not aspirational.** A pre-commit hook blocks committing any project or topic page over about 150 lines, so an oversized page can't slip into a commit unnoticed and turn up three months later at 800 lines. 4. **Runtime enforcement is the next level up, if I ever need it.** A tool hook that intercepts reads and warns or blocks on a real bulk-read pattern, or just counts files touched per turn. Worth building once I actually see the behavior, not pre-built against a hypothetical. 5. **Cost stays visible after the fact.** The log entry can note roughly how many pages a task touched, so a periodic pass can flag drift the same way it flags stale projects.

Privacy

Anything with personal data, customer data, or other sensitive material goes in a `private/` subfolder, which is gitignored: never pushed to GitHub, so it never reaches mobile or cloud agents either. It stays local, backed up however I back up my machine (not by GitHub). Everything else is plain markdown, safe to sync everywhere.

Cross-tool and mobile access

Each tool keeps its own native config format (Claude Code uses `.claude/`, Codex uses `.codex/`, Cursor uses `.cursor/`), but they all point at the same shared conventions and the same `AGENTS.md` contract. The instructions get written once, and every tool follows the same rules. Because the repo lives on GitHub:

- Claude on the web or on mobile can open it directly.
- Codex's cloud mode can check it out the same way.
- Any tool with a GitHub-aware agent mode can point at it the same way. You're not limited to your desktop.

One honest caveat, and it's why Grok isn't in the folder list above: a tool needs some way to read and write GitHub (a native connector, or the ability to run git) to fully participate. Grok doesn't yet, so for me it's read-only: I paste `AGENTS.md` plus a project page in when I want it caught up, but it can't write back on its own. *(Update 2026-10-26: this caveat has since shrunk; a chat tool now writes through the inbox. See "Two months in" below.)*

What this costs, in tokens

Roughly nothing to run (it's markdown and git), but it isn't literally free. A task following the reading protocol pays for `AGENTS.md` plus `HOME.md` once, plus whatever handful of linked pages the task actually needs (each capped around 150 lines, so maybe 500 to 800 tokens apiece, call it 1,500 to 3,000 more for two to four relevant links). The first part is the one to watch, because it's paid on every task and it drifts. Mine ran about 4,200 tokens combined when I first published this (a rough four-characters-per-token estimate). Two months later it had crept to about 4,700, almost all of it in `HOME.md`, whose recent-log list had grown to fifty entries. Capping that list at the newest ten brought the pair down to about 3,000. So the per-task overhead is roughly 4,500 to 6,000 tokens of added input, cold, and the growth vector is the index, not the pages. That's the whole point of not bulk-reading the repo. Without that rule, you'd pay for the entire wiki every session instead of a slice of it, and the cost would grow as the wiki grows instead of staying roughly flat.

Two things soften this further. Prompt caching means repeat reads of `AGENTS.md` and `HOME.md` within the same session are billed at a fraction of full price. And the write side (a log entry plus a small page update) is only a few hundred tokens of generation; the git commit and push cost nothing, they're shell commands. The real lever on cost isn't the hub's existence, it's holding the line on the two policies above: keep pages small (now enforced by a hook, not just a convention), and don't let an agent read everything "to be safe."

Try it yourself

Pick whichever path fits how you work.

Option 1: clone the repo

I've published a working skeleton at github.com/eytaniam/public/tree/main/agent-hub-template. It already has the `AGENTS.md` contract, the folder structure, the pre-commit hook, the librarian, the write-back skill, and per-tool config for Claude Code, Codex, and Cursor in place. It has a `CHANGELOG.md` too, so if you cloned an earlier version you can see what moved. Grab it, run `git init`, push it as your own repo, and start writing.

One thing to know before you assume it's fully wired up: the auto-load only fires while your agent's working directory is inside the cloned folder. If you're happy doing hub-related work as its own dedicated session, that's already enough, no extra step needed. If you want every session, in any project folder, to know the hub exists, hand your agent this prompt once rather than clicking through settings yourself. All three tools I use can make this change themselves: Claude Code and Codex just edit a plain text file, and Cursor has its own tool for adding a global rule.

```
Add a pointer to my agent hub to your own global configuration, not
just this
project's:

- If you're Claude Code: add it to ~/CLAUDE.md (my user-level file, not
this
  repo's), under a new "## Agent hub" heading.
- If you're Codex: add it to ~/.codex/AGENTS.md (my global instructions
  file), under a new heading.
- If you're Cursor: add it as a new global User Rule, not a project rule
  under .cursor/rules/ (that only fires in this one repo). Name it
  "Agent hub" and set it to Always Apply, not Apply Intelligently.

The pointer text:
"My agent hub lives at /Users/<you>/agent-hub. When a task touches an
ongoing project, a prior decision, or anything worth remembering across
sessions, read AGENTS.md there and follow it: start from HOME.md, then
write back at the end of meaningful work. Ask before committing and
pushing hub changes."

Show me the exact change before you save it -- this is a global file
that
affects every future session, not just this one.
```

Two things worth knowing about that prompt. Don't let it substitute Claude Code's separate persistent-memory feature for the file edit above: I tried that first, and it turned out to be scoped per project directory, not global, so a pointer saved there only covers sessions started from that same folder. And the pointer text uses an absolute path (`/Users/you/agent-hub`), not `~`, since `~` doesn't reliably expand inside every tool call.

Option 2: prompt an agent you already have

If you'd rather have your own Claude, Codex, or similar set this up for you from scratch, hand it this prompt as-is:

```
Set up a personal "agent hub": a git repo that serves as shared long-
term memory
across all the AI agents I use, so nothing I learn in one session is
lost and I
can access it from my phone too.

Create a new local folder and git repo with this structure:

  AGENTS.md          — root contract file (see contents below)
  CLAUDE.md          — symlink to AGENTS.md
  HOME.md            — index page, frontmatter: type: topic, status:
active, updated: <today>
  projects/          — empty for now, one .md page per ongoing project
later
  topics/            — empty for now, cross-cutting concept pages later
  docs/              — empty for now, finished memos/specs go here
  work/              — empty for now; work/<project>/ folders are
created as work starts
  log/               — empty for now, append-only dated journal entries
  inbox/             — empty for now, quick-capture drops to be filed
later
  .gitignore         — ignore: .DS_Store, work/*/private/,
work/private/, private/

Write AGENTS.md with this contract (adapt wording to taste but keep the
mechanics):

1. Every page is a small wiki page: frontmatter (type: project | topic |
doc | log |
   inbox, status, updated: YYYY-MM-DD) followed by content. Cross-
reference other pages with [[wiki-links]] — a link
   resolves to a unique filename anywhere in the repo.
2. Reading protocol for any agent starting a task: read HOME.md first,
then follow
   only the links relevant to the task. Never bulk-read the whole repo.
3. Writing protocol for any agent finishing a task: log the work to
   log/YYYY-MM-DD-short-slug.md (what happened, key decisions + why,
what was
   produced and where); update the relevant projects/ page to reflect
the new
   current state; link liberally, including to pages that don't exist
```

```
yet;
   update HOME.md if a new page was created; then ask me before running
   git add -A && git commit -m "log: <summary>" && git push.
4. Freshness labels on facts inside project pages, not just a page-level
date:
   "Current as of YYYY-MM-DD:", "Decision (YYYY-MM-DD):",
   "Open question as of YYYY-MM-DD:", "Historical note (YYYY-MM-DD):",
   "Needs verification:".
   When a fact is superseded or no longer true, close it with an end
date and
   a pointer instead of deleting it, e.g.
   "Decision (2026-06-01 → 2026-08-10, superseded by [[new-page]]):".
   Treat anything unlabeled, open-ended, or >30 days old as needing re-
verification.
5. Sensitive content (PII, customer data, anything you wouldn't want on
GitHub)
   goes in a private/ subfolder inside the relevant work/<project>/
folder —
   gitignored, local-only, never pushed or synced to mobile/cloud
agents.
6. Filenames: kebab-case, unique across the whole repo (this is what
makes
   [[links]] unambiguous without a path).
7. Pages stay small — split anything that grows past ~150 lines into a
   distilled current-state page + a log entry holding the history. Make
this
   mechanical, not aspirational: add a pre-commit hook (tracked in the
repo,
   e.g. .githooks/pre-commit, enabled via `git config core.hooksPath
   .githooks` so it travels with clones) that fails the commit if any
   projects/*.md or topics/*.md file is over ~150 lines.
8. There is deliberately no "read the whole repo" script or skill —
don't
   build one later for convenience. If HOME.md ever stops being a good
   enough index to answer "where do I look," that's the actual problem
to
   fix, not a reason to add a bulk-read shortcut.
9. Other sessions, including mobile or cloud ones, may write to this
repo
   too — git pull before making changes, not just before the final push,
to
```

```
   avoid a rejected push or a conflict.

Then:
- Set up per-tool config so every agent I use (Claude Code, Codex,
Cursor,
  or others) reads AGENTS.md automatically at the start of a session,
and
  follows the same read/write protocol above. Use each tool's native
  convention for this (Claude Code auto-reads CLAUDE.md at repo root;
Codex
  and Cursor auto-read AGENTS.md there).
- Package the writing protocol as a skill named hub-log, so the write
step
  has a trigger and not just a rule: put the same SKILL.md in
  .claude/skills/hub-log/, .agents/skills/hub-log/, and
.cursor/skills/hub-log/.
  Its description should fire on "log this to the hub" and proactively
at
  the end of any substantial task; its body is step 3 above, spelled
out.
- Add a global pointer too, not just the project-level file: edit
  ~/CLAUDE.md (Claude Code), ~/.codex/AGENTS.md (Codex), or add a global
  User Rule set to Always Apply, not Apply Intelligently (Cursor) — for
  whichever tools I use — so a session started from any other project
  folder still knows this hub exists. Use an absolute path, not ~, and
  show me the exact change before saving it, since these are global
files
  that affect every future session, not just this one.
- Create a GitHub repo and push this as the origin, so it's reachable
from
  mobile (Claude web/mobile, Codex cloud, or any other GitHub-aware
agent).
- Optionally, define a small "librarian" helper (a subagent/config,
whatever
  the tool supports) that on request: fixes broken [[links]], files
inbox/
  items into the right pages, splits overgrown pages into log/ +
distilled
  page, refreshes HOME.md, and flags projects untouched for 30+ days.

Confirm the structure with me before making the first commit, and ask
```

```
which
tools I actually use so you only wire up config for those.
```

Whichever path you take, three things are worth knowing going in:

- **The write step is what keeps it current.** Log, then update the page, then commit, at the end of every session. Nothing enforces it mechanically, but it no longer relies on the agent remembering either: it's a skill with a trigger now (see "Two months in," below).
- **Not every tool can write back equally well.** Anything with real git and file access can fully participate. A chat-only tool can read the wiki through a GitHub connector or a pasted `AGENTS.md` plus a project page, and it can write by drafting a log entry that you drop in `inbox/`. The push itself still has to come from a tool that can push.
- **Start small.** One `HOME.md`, one real project page, and the logging habit matter more than a fully built-out folder tree with nothing in it.

That's genuinely the system I run every day, across every machine I work from. If you set up your own version of it, I'd like to hear what you change.

Two months in: what changed

Added 2026-10-26. A system like this should visibly evolve, so rather than quietly rewriting the post I'm keeping the recipe above as it was, fixing only the parts that would trip you up (marked inline), and recording what changed here. The template has a `CHANGELOG.md` that mirrors this section. Each item is a problem I hit, what I changed, and why.

Standing context is the cost to police, not wiki size. The problem: the wiki stayed cheap, but the two files every agent loads on every task did not. `HOME.md` had quietly doubled, and `AGENTS.md` had accumulated rules that belonged to specific projects (how to handle a to-do, which deck template to use). The change: `AGENTS.md` carries only the contract, page-specific rules live on the page that owns them with a one-line pointer from the contract, and the `HOME.md` recent-log list is capped at the newest ten entries (the full journal is one `ls log/` away). The why: the wiki is loaded on demand, but the contract and the index are loaded unconditionally, so they are the only place where growth compounds. This is also my answer to the "delete 90% of your AI setup" advice going around: most of it doesn't apply to a workspace you load on demand, and the part that does apply is exactly this.

The write step is a skill now, not a hope. The problem: I'd told you above to have your agent "nag itself" to log work, which is to say I had no mechanism. The change: the writing protocol is packaged as a skill called `hub-log`, with a description that fires on "log this to the hub" and proactively at the end of any substantial task, and a body that spells out the log-entry shape (what happened, decisions with their reasons, outputs and where they live). The same file sits in `.claude/skills/`, in `.agents/skills/` (the tool-neutral location Codex reads), and in `.cursor/skills/`, so it travels with every clone. The why: a rule in a contract competes with everything else in context; a skill with a trigger is loaded at the moment it's needed.

Chat-only tools write through the inbox. The problem: the Grok caveat above, generalized. Any tool that can read but not push was locked out of the write side. The change: a chat tool gets a four-line preference (read `HOME.md` before answering questions about my projects), reads the hub through a GitHub connector, and when a conversation produces a decision worth keeping it drafts a log entry in the `log/YYYY-MM-DD-slug.md` format. I drop that into `inbox/`, or hand it to a session that can push, and the librarian files it. The why: the inbox already existed for exactly this shape of thing, raw drops waiting to be filed. It just hadn't occurred to me that an agent could be the one dropping.

The hub is a routing layer, not a mirror. The problem: the privacy section covers what goes in `private/`, but says nothing about what shouldn't be in the hub at all, and the hub started attracting copies of things that live better elsewhere. The change: a short "where finished work goes" section in the contract. Memos and specs go in `docs/`; a doc that gets reviewed in a shared drive keeps its markdown source in the hub with the drive link as `canonical:` in the frontmatter, and says which copy wins; decks, PDFs, and large HTML stay where they are and the hub gets a pointer; a large documentation corpus gets a routing page that says where to look, never a copy. The why: a mirror is a stale second source of truth by its second week. The hub holds decisions and pointers; the thing itself stays at its source.

Two smaller notes. First, on the tools' own memory features: I'm leaving them on. Stable preferences (how I like things written, where the hub lives) belong in the tool's memory; project facts belong in the hub, where they carry freshness labels; and a sensitive conversation gets memory turned off for that chat. Second, a trap if you read the hub on your phone through GitHub: GitHub Pages on a personal account is public even when the repository is private. Read the repo directly, or generate a `README.md` with an action. Don't turn on Pages.